GSE Management Unit (GSEMU)

Hardware and Software Reference

Ken Overton

KJO Liquid Rocket Motor Control System

Version: 0.6

Date: September 2026

This document describes the hardware, software architecture, Fill valve control, and GPIO handshake protocol of the GSE Management Unit. The GSEMU is mounted on the launch pad and controls the propellant fill valve and the umbilical quick-release servo.

Contents

1 Hardware Overview

1.1 Component Summary

Component	Part	Interface	Notes
Microcontroller	Adafruit Feather M0 (ATSAMD21G18A, 48 MHz)	—	
OLED Display	SH1107, 128×64, 1.12"	I^2C	
GPIO Expander	MCP23X17 (16 I/O)	I^2C	Buttons, AUX_IO
PWM Servo Wing	Adafruit 8-channel wing (PCA9685)	I^2C	Fill valve servo
Real-Time Clock	PCF8523	I^2C	
SD Card	Standard SPI SD socket	SPI	
Battery	3.7 V LiPo	—	

1.2 MCP23X17 GPIO Expander Pin Map

Pin Name	Direction	Function
Button A (QR release)	IN	Hold-to-release QR latch (servo open while held, hold when released)
Button B (fill close)	IN	Locally command Fill valve closed
Button C (fill open)	IN	Locally command Fill valve open
`AUX_IO_1` (`GSEMU_LINK_SENSE_EIO`)	IN (INPUT_PULLUP)	Umbilical link-disconnect sense; wired to EMU chassis GND (LOW = connected, HIGH = separated). Fill valve CMD/STATUS formerly carried here has moved to CAN.
`AUX_IO_2`	—	Retired/unused (formerly `FILL_VALVE_STATUS_EIO`).
`AUX_IO_3` (`QR_SENSE_EIO`)	IN (INPUT_PULLUP)	Umbilical QR physical-connection sense; wired to EMU chassis GND (LOW = connected, HIGH = separated). QR release itself is commanded over CAN (**`CAN_QR_RELEASE`**), not this line.
`AUX_IO_4`	—	Retired/unused (formerly `RELEASE_STATE_EIO`; its sense function consolidated onto `QR_SENSE_EIO`).
`AUX_IO_5`	—	Retired/unused (formerly `LAUNCH_ENABLE_EIO`).
`AUX_IO_6`	—	Retired/unused (formerly `REMOTE_START_EIO`).

1.3 Hardware Block Diagram

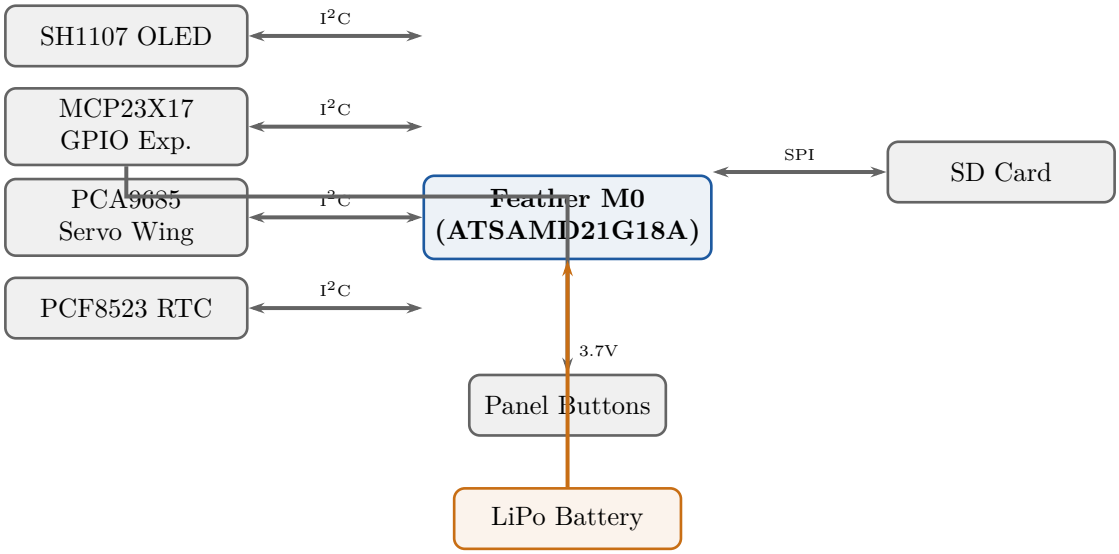

Figure 1: GSEMU hardware block diagram.

2 Software Architecture

2.1 Module Summary

Module	Location	Responsibility
`main.cpp`	local	Setup, loop, button handling
`KJO_Valve`	local	Base valve: servo drive, pot feedback (Fill valve open/close is CAN-commanded; see §3)
`KJO_QR_Servo`	local	Servo-only actuator base class (time-based position)
`KJO_QR_Slave`	local	QR release actuator: latches **`CAN_QR_RELEASE`**, reads connection-sense (`AUX_IO_3`), local override
`KJO_GPIO`	local	MCP23X17 pin assignments, QR servo constants
`KJO_GSE_Display`	local	SH1107 OLED display, SD log helpers
`KJO_Logging`	local	SD log file constants
`KJO_System_Config`	shared	Version, platform constants

2.2 Class Hierarchy

`Fill_Valve` is a plain `Valve` instance – there is no derived class for it. The `Valve` base class provides servo drive and potentiometer feedback; `open()`/`close()` are called directly from `Check_CAN()` (see §3).

`QR_Servo` (base) ⟵ `QR_Slave` (derived).

`QR_Servo` provides servo drive via the PCA9685 with time-based motion completion detection (no potentiometer). `QR_Slave` adds a permanent release-commanded latch (set by **`CAN_QR_RELEASE`**), physical separation detection via `AUX_IO_3` (`QR_SENSE_EIO`), and a local Button A override (`localRelease()` / `localHold()`).

2.3 Main Loop Control Flow

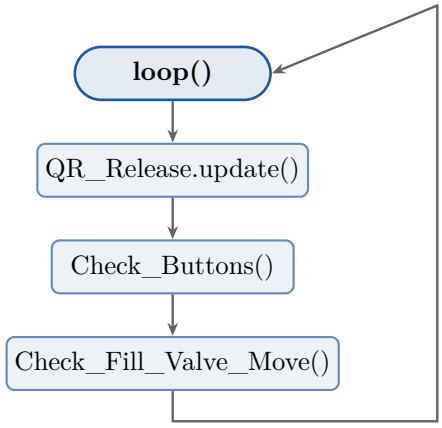

Figure 2: GSEMU main loop control flow.

3 Fill Valve Control

3.1 CAN-Commanded Operation

The wired `AUX_IO_1`/`AUX_IO_2` CMD/STATUS handshake this section formerly described has been retired; there is no `Slave_Valve` class in the current firmware. `Fill_Valve` is a plain `Valve` instance, opened/closed directly on EMU's CAN command. GSEMU has no fill state machine of its own – EMU owns tare/poll/target tracking for the whole fill session.

1. `Check_CAN()` receives **`CAN_OPEN_FILL_VALVE`** or **`CAN_CLOSE_FILL_VALVE`** and calls `Begin_Fill_Move()`, which issues `Fill_Valve.open()`/`close()` immediately.
2. The CAN response is *deferred* – `Check_Fill_Valve_Move()` (called every `loop()`) sends it once `Fill_Valve.isStopped()` confirms the move, or `MOVE_TIMEOUT`+100 ms elapses, reporting the resulting position either way.
3. This same mechanism is also reachable directly via RCU_UNIT_TEST's **`OPEN_FILL_VALVE`**/**`CLOSE_FILL_VALVE`** bench-test commands, for exercising the valve without a full fill session.

3.2 Local Button Override

Three panel buttons on the MCP23X17 provide local control during ground operations:

Button	Action
A (QR Release)	**Hold-to-release**: calls `localRelease()` on press; servo opens while button held. Calls `localHold()` on release; servo returns to hold position. Event logged to SD card.
B (Fill Close)	Drive Fill valve servo to fully closed position (falling-edge triggered).
C (Fill Open)	Drive Fill valve servo to fully open position (falling-edge triggered).

Warning

Button A activates the umbilical quick-release servo. The latch opens for as long as the button is held, and releases pressurised N_2O when the umbilical separates. Ensure all personnel are clear and the pad is safe before pressing Button A. Buttons B and C bypass EMU command authority; use for ground testing only.

3.3 Fill Valve Position Query

EMU (or RCU_UNIT_TEST) can poll the Fill valve's live position at any time via **CAN_GET_FILL_VALVE_POSITION**, answered immediately with `Fill_Valve.getPositionPercent()` – independent of any in-progress open/close move.

4 Umbilical Quick-Release (QR_Slave)

4.1 Overview

The umbilical quick-release servo physically separates the umbilical connector from the rocket airframe at launch. The GSEMU drives the servo; release is commanded over CAN (**CAN_QR_RELEASE**, fire-and-forget from EMU) rather than a dedicated wire. Release is one-shot: once `commandRelease()` is called, the servo opens and stays open – there is no remote command back to "hold." A spring-loaded latch, once triggered, cannot be electrically un-released; only physical re-latching (and/or the local front-panel override below) changes it.

4.2 Hardware

The QR servo is driven by a channel on the PCA9685 Servo Wing. Unlike the Fill valve servo, there is no potentiometer feedback; position is determined entirely by elapsed time (`QR_SERVO_MOVE_MS`).

One MCP23X17 expansion I/O line is used for connection sensing:

- `AUX_IO_3` (`QR_SENSE_EIO`) — INPUT_PULLUP, wired to EMU chassis GND. LOW = umbilical physically connected (EMU GND pulls it LOW); HIGH = umbilical has separated.

Warning

`QR_SERVO_PWM_HOLD` and `QR_SERVO_PWM_OPEN` are placeholder values in the current firmware. They **must** be calibrated on the bench before any live operation. Verify servo travel direction and endpoint positions before connecting the umbilical.

4.3 Servo Behaviour

`QR_Slave::update()` polls every `loop()` iteration:

- Local override active OR release commanded, and servo not already opening/open ⇒ `openServo()`.
- Otherwise, and servo not already holding/held ⇒ `holdServo()`.

Physical separation (state line LOW → HIGH) is detected and latched; `isSeparated()` returns true once separation has been confirmed. `isCurrentlyConnected()` instead returns the live,

non-latching state – used where behaviour must resume normally on reconnect (e.g. the Fill Valve safety interlock in `main.cpp`), unlike the permanently-latched `isSeparated()`.

4.4 Local Button A Override

Button A on the GSEMU front panel engages a local override that bypasses the release-commanded latch and forces the servo open, independent of whether CAN release has been commanded. `Check_Buttons()` calls:

- `QR_Release.localRelease()` on button press (falling edge).
- `QR_Release.localHold()` on button release (rising edge).

The override is cleared on button release; the servo then follows the release-commanded latch (open if `CAN_QR_RELEASE` was ever received, hold otherwise) on the next `loop()` iteration.

4.5 QR_Slave API

Method	Action
`begin()`	Configure the state pin as INPUT_PULLUP; command servo to HOLD
`update()`	Drive servo per local override / release-commanded latch; detect separation
`commandRelease()`	Latches the release command (permanent – never cleared); called from `Check_CAN()`'s `CAN_QR_RELEASE` case
`isSeparated()`	Latched: true once the state line has gone HIGH (physical separation confirmed)
`isCurrentlyConnected()`	Live, non-latching: true while the state line currently reads LOW
`localRelease()`	Set local override; servo opens on next `update()`
`localHold()`	Clear local override; servo follows the release-commanded latch on next `update()`

5 OLED Display

The SH1107 OLED provides local status when no serial console is available. `KJO_GSE_Display` provides:

- `scrollMessage(msg)`: scrolls a text message across the display.
- `displayValveStatus(pct)`: shows the current valve open percentage.
- `displayLogEntry(msg)`: echoes SD log entries to the display.

6 SD Logging

Log files are stored in the root of the SD card as `Log_N.txt` (N is auto-incremented to avoid overwriting existing files). Each record is time-stamped (PCF8523) and logs CMD/STATUS transitions, servo position changes, button presses, and system events.

7 Parameters & Flags

This section collects the tunable operational parameters and feature flags compiled into this firmware image, together with the source file where each is defined. It is meant as a single reference point when tracing or changing a timing, threshold, or calibration value. Pin/channel wiring assignments, CAN command opcodes, and packet byte layouts are documented separately (the Hardware Overview pin map and the Command Set sections above) and are not repeated here. Parameters marked *(shared)* are defined in `KJO_Shared_Libraries` and must agree with the same constant used on the other board(s) that consume it.

Fill Valve Servo Drive & Motion Detection

Parameter	Value	Description	Source File
`SERVO_FREQUENCY`	1100 Hz	PWM frequency driven above the standard 50 Hz to compress the 5-turn servo's usable travel into a smaller PWM-count range, improving position resolution.	`KJO_Valve.h`
`SERVO_RANGE_MIN`	500 μs	Servo pulse width at the lower mechanical travel limit.	`KJO_Valve.h`
`SERVO_RANGE_MAX`	2500 μs	Servo pulse width at the upper mechanical travel limit.	`KJO_Valve.h`
`SERVO_FULL_RANGE_DEGREES`	1800°	Total mechanical rotation of the 5-turn servo across its full pulse-width range.	`KJO_Valve.h`
`SERVO_PINION_TEETH`	16	Tooth count on the servo output pinion, used to compute the servo-to-valve gear ratio.	`KJO_Valve.h`
`VALVE_GEAR_TEETH`	60	Tooth count on the valve actuator gear, used with `SERVO_PINION_TEETH` to compute the gear ratio.	`KJO_Valve.h`
`VALVE_TURN_DEGREES`	90°	Ball-valve rotation from fully open to fully closed.	`KJO_Valve.h`
`VALVE_MOVE_TIMEOUT_MS` (alias `MOVE_TIMEOUT`)	2500 ms	Maximum time allowed for a commanded Fill Valve move to complete before it's treated as failed/timed out; shared with EMU since EMU's own CAN-relay timeout for this valve is sized off this value.	`KJO_Valve_Timing.h` (shared)
`VALVE_POSITION_AVERAGING_COUNT`	4	Number of ADC reads averaged per position sample to reduce potentiometer noise.	`KJO_Valve.h`
`VALVE_MOVING_THRESHOLD`	2 counts	Minimum position-sensor delta between samples required to consider the valve still moving.	`KJO_Valve.h`
`VALVE_POSITION_DEAD_BAND`	15 counts	Position tolerance used to declare the valve "open," "closed," or "at target."	`KJO_Valve.h`
`MOVE_DETECT_WAIT`	30 ms	Minimum interval between position samples inside `isMoving()`'s velocity check.	`KJO_Valve.h`
`MOVE_AFTER_START_WAIT`	60 ms	Blocking delay after issuing a new PWM target, giving the servo time to start moving before the first `isMoving()` sample.	`KJO_Valve.h`
`USE_MAPPING`	`true`	Feature flag selecting angle-geometry-based valve position mapping over simple linear PWM interpolation.	`KJO_System_Config.h` (shared)

Fill Valve Calibration (Travel Endpoints & Ball-Valve Geometry)

Parameter	Value	Description	Source File

FILL_VALVE_PWM_OPEN	2329	Servo PWM count corresponding to the Fill Valve's fully-open position.	KJO_Valve.h
FILL_VALVE_PWM_CLOSE	3918	Servo PWM count corresponding to the Fill Valve's fully-closed position.	KJO_Valve.h
FILL_VALVE_POS_OPEN	1310 counts	Potentiometer reading corresponding to the Fill Valve's fully-open position.	KJO_Valve.h
FILL_VALVE_POS_CLOSED	3004 counts	Potentiometer reading corresponding to the Fill Valve's fully-closed position.	KJO_Valve.h
FILL_VALVE_BALL_DIAMETER	0.520 in	Ball outer diameter used to compute the angle-based flow-restriction mapping.	KJO_Valve.h
FILL_VALVE_BORE_DIAMETER	0.275 in	Ball bore diameter used in the same geometric flow-mapping calculation.	KJO_Valve.h
FILL_VALVE_THROAT_DIAMETER	0.275 in	Valve body throat inner diameter used in the same geometric flow-mapping calculation.	KJO_Valve.h

Umbilical Quick-Release (QR) Servo

Parameter	Value	Description	Source File
QR_SERVO_PWM_HOLD	2400	PWM count for the latch-engaged (held/connected) servo position, bench-calibrated March 2026.	KJO_GPIO.h
QR_SERVO_PWM_OPEN	2800	PWM count for the latch-released (open) servo position.	KJO_GPIO.h
QR_SERVO_MOVE_MS	1000 ms	Time allotted for the QR servo to complete a full stroke between hold and open.	KJO_GPIO.h
QR_TEST_RELEASE_DURATION_MS	3000 ms	Bench-test-only dwell time the QR servo stays released before automatically returning to hold (**QR_TEST_RELEASE**, RCU_UNIT_TEST); never applied to the permanent production release latch.	main.cpp

LCO Ignition Sense & Battery Monitoring (Analog Subsystem)

Parameter	Value	Description	Source File
AD_BASE_SCALE	0.00199144777 V/count	ADS1015 volts-per-count scale factor at GAIN_ONE (4.096 V / 2048 counts); identical for all boards using this gain setting.	KJO_Analog.h
GSEMU_LIPO_SCALE	2.79111	Voltage-divider scale factor converting raw ADC counts to 2S LiPo battery voltage; a placeholder seeded from EMU's divider, pending measurement of GSEMU's actual divider resistors.	KJO_Analog.h
LCO_SENSE_THRESHOLD (alias AD_AUX_THRESHOLD)	251 counts	Digital HIGH/LOW threshold (∼0.5 V post-divider) for the AUX/LCO analog input; bare unsigned comparison, not abs(), since ADS1015 input-protection diodes make a reversed-polarity reading unreliable to detect this way. Shared with EMU's Airstart sense so both boards interpret the same signal identically.	KJO_LCO_Sense.h (shared)
LCO_SENSE_ADS_CHANNEL (alias AD_AUX_CHANNEL)	0	ADS1015 channel carrying the AUX/LCO sense signal; primarily a wiring/channel assignment, centralized here only to guarantee EMU and GSEMU agree on which channel carries this cross-board safety signal.	KJO_LCO_Sense.h (shared)

BATTERY_DISPLAY_INTERVAL_MS	10000 ms	Interval at which the GSEMU 2S LiPo battery voltage is re-read, displayed, and logged.	KJO_Analog.h

CAN Communication Timing

Parameter	Value	Description	Source File
CAN_BAUDRATE	1,000,000 bps	CAN bus bit rate, raised from a prior 250 kbps; validated against ISO 11898-2's 1 Mbps/40 m limit for this installation's 16 ft maximum run.	KJO_CAN_Command_Defs.h (shared)

SD Logging

Parameter	Value	Description	Source File
LOG_FILE_NAME_BASE	"Log_"	Base filename prefix for sequentially-numbered SD log files (e.g. Log_0.txt), written flat to the SD card root.	KJO_Logging.h

OLED Boot/Status Scroll Display

Parameter	Value	Description	Source File
MAX_LINES	8	Number of most-recent status lines retained in the scrolling OLED display's circular message buffer.	KJO_Status_Display.cpp (shared)

8 Version History

Version	Date	Notes
0.1	March 2026	Initial release.
0.2	March 2026	Add umbilical QR release subsystem (QR_Servo, QR_Slave, AUX_IO_3 protocol).
0.3	March 2026	Level-based QR protocol (CMD LOW=release, HIGH=hold); add RELEASE_STATE_EIO (AUX_IO_4), LAUNCH_ENABLE_EIO (AUX_IO_5), REMOTE_START_EIO (AUX_IO_6); Button A reassigned to hold-to-release QR latch; Button C now opens Fill valve; `QR_Slave` API adds `localRelease()`, `localHold()`, `isSeparated()`; `Check_AUX_Input()` drives gated Remote Start output.
0.4	March 2026	SD log files moved to SD card root (`Log_N.txt`); subdirectory path constant (`LOG_FILE_FOLDER`) removed from `KJO_Logging.h` and `KJO_GSE_Display`.
0.5	September 2026	Add new §7 Parameters & Flags reference section: tunable timing, threshold, and calibration constants across the Fill valve, QR servo, LCO sense, and CAN subsystems, with source-file attribution for each.
0.6	September 2026	Corrected stale wired-GPIO descriptions left over from the CAN redesign: §3 Fill Valve Control rewritten (no `Slave_Valve` class exists; `Fill_Valve` is CAN-commanded directly from `Check_CAN()`); §4 Umbilical Quick-Release (`QR_Slave`) rewritten to describe the permanent **`CAN_QR_RELEASE`** latch instead of a level-based CMD line; corrected 5 of 6 `AUX_IO` pin-map rows to match (only the umbilical/link-sense functions survive); fixed the Module Summary and Class Hierarchy (§2) and Main Loop Control Flow diagram (§2.3) to match.